gemm_main_fsm

Main FSM và bộ lập lịch descriptor

Port list, job, ba engine Đọc/Tính/Ghi, chọn bank, lỗi, soft reset

Bộ tăng tốc GEMM INT8 trên Chipyard SoC

Module	`gemm_main_fsm` (instance `u_main_fsm` trong `gemm_accel_top`)
Module con	`gemm_fsm_ctrl`, `gemm_job_check`, `gemm_rd_engine`, `gemm_cp_engine`, `gemm_wr_engine`, `gemm_bank_sb`
Cấu hình	Đặc tả bản đầy đủ: `HAS_CHAIN` = 1, `HAS_PINGPONG` = 1. Với tham số bằng 0, cùng RTL rút về hành vi của mục 1 – 6 spec hệ thống (mục 2)
Tuân theo	Spec hệ thống v1.0: mục 3 (TSK-*, REG-11, REG-12, bảng mã lỗi), mục 4 (pha, lỗi, RST-01 … RST-09), mục 5.2 (FSM-01 … FSM-06), mục 7 (BS-*, DSC-01 … DSC-11, PP-01 … PP-10)
Kèm theo	`gemm_sched_model.py`: mô hình cycle-accurate của bộ lập lịch, có dữ liệu thật; kiểm kết quả DRAM so với thực thi tuần tự, thứ tự pha, chọn bank, lỗi

Mục lục

1 Chức năng **3**

2 Tham số **3**

3 Port list **4**

4 Vi kiến trúc **5**

4.1 Job . . . 6

5 Dựng và kiểm tra job **6**

5.1 Chế độ đơn . . . 6

5.2 Chế độ chuỗi . . . 7

6 Bank và cấp quyền bắt đầu pha **7**

6.1 Chọn bank . . . 7

6.2 Bộ đếm chiếm dụng . . . 8

6.3 Cấp quyền bắt đầu pha . . . 8

7 Engine **9**

7.1 Lệnh DMA . . . 9

7.2 Engine Đọc . . . 9

7.3 Engine Tính và engine Ghi . . . 10

8 Điều khiển tác vụ **10**

9 Timing **11**

9.1 Ví dụ lịch chạy . . . 12

10 Reset, hiện thực, kiểm chứng **12**

A Nối dây nội bộ trong `gemm_accel_top` **13**

1 Chức năng

Main FSM là module duy nhất ra lệnh cho DMA và array. Mỗi phép GEMM được biểu diễn bằng một **job** (lấy từ bản chụp thanh ghi ở chế độ đơn, hoặc từ một descriptor ở chế độ chuỗi). Ba engine độc lập đưa job qua các pha: **Đọc** (FETCH, kiểm tra, LOAD_*), **Tính** (COMPUTE) và **Ghi** (STORE_C). Bộ chấm điểm bank quyết định khi nào một pha được bắt đầu, nên khi có ping-pong ba pha của ba job liên tiếp chạy chồng nhau mà không bao giờ đụng cùng một bank.

Bảng 1.1: Chức năng

ID	Yêu cầu
FSM-F01	Chế độ đơn (START): một job, các pha CHECK_CFG → LOAD_BIAS (nếu `POST_EN`) → LOAD_A → LOAD_B → COMPUTE → STORE_C, tuần tự, đúng mục 4 spec hệ thống (BS-03)
FSM-F02	Chế độ chuỗi (START_CHAIN): mỗi descriptor là một job, chạy FETCH → kiểm tra → LOAD_BIAS → LOAD_A → LOAD_B (trừ khi `REUSE_B`) → COMPUTE → STORE_C (DSC-02); chuỗi dừng ở descriptor có `LAST` = 1
FSM-F03	Với `HAS_PINGPONG` = 1, các pha của các job khác nhau chạy chồng theo PP-01 … PP-08. Với `HAS_PINGPONG` = 0, mọi pha dùng bus hoặc scratchpad loại trừ nhau (hình 7-2a spec hệ thống)
FSM-F04	Nội dung DRAM sau một tác vụ DONE giống hệt thực thi tuần tự từng job, với mọi cấu hình tham số và mọi thời gian trả lời của bus (PP-08, FUN-07)
FSM-F05	FSM báo kết thúc tác vụ cho Register File bằng đúng một xung done hoặc một xung error; điều phối soft reset; cấp bộ đếm `DESC_DONE_CNT`, thanh ghi `ERR_DESC_ADDR` và tín hiệu đếm hiệu năng
FSM-F06	FSM cấp cấu hình của job đang tính cho array và VPU, và bank đang tính cho scratchpad. Bản chụp của Register File chỉ đi tới FSM

THAY ĐỔI PORT NỘI BỘ SO VỚI BẢNG 5-0 CỦA SPEC HỆ THỐNG
Theo BS-04: (1) bản chụp cấu hình chỉ đi tới FSM; array và VPU nhận cấu hình của job đang tính từ FSM (`o_comp_*`); (2) DMA có hai kênh với lệnh mang đủ tham số (địa chỉ, số hàng, số word mỗi hàng, stride, bank) thay vì tự đọc bản chụp; (3) scratchpad nhận bit chọn bank. Các cam kết hành vi của mục 5 spec hệ thống không đổi.

2 Tham số

Bảng 2.1: Tham số

Tham số	Mặc định	Ý nghĩa
`P`	8	Kích thước array; 4 hoặc 8. Chỉ dùng cho `HW_CONFIG` và assertion
`HAS_CHAIN`	1	1: có chế độ chuỗi. 0: START_CHAIN kết thúc với mã lỗi 5 (REG-04); engine Đọc không có FETCH
`HAS_PINGPONG`	1	1: hai bank mỗi buffer, các pha chạy chồng. Đòi hỏi `HAS_CHAIN` = 1
`NB`	2	localparam = `HAS_PINGPONG` ? 2 : 1, số bank mỗi buffer

Bảng 2.2: Hành vi theo cấu hình

Cấu hình	Hành vi
`HAS_CHAIN` = 0, `HAS_PINGPONG` = 0	Bản cơ bản của mục 1 – 6. Mọi job có bank 0; chỉ một job mỗi tác vụ
`HAS_CHAIN` = 1, `HAS_PINGPONG` = 0	Chuỗi descriptor, các pha tuần tự tuyệt đối (quy tắc SEQ, mục 6.3)
`HAS_CHAIN` = 1, `HAS_PINGPONG` = 1	Bản đầy đủ: chuỗi descriptor, ping-pong
`HAS_CHAIN` = 0, `HAS_PINGPONG` = 1	Không hợp lệ: chặn lúc elaboration

3 Port list

Bảng 3.1: Port của `gemm_main_fsm`

Port	Rộng	Chiều	Nối với	Mô tả
Clock, reset, soft reset				
`clk`, `rst_n`	1, 1	In	top	Clock; reset đã đồng bộ (`rst_n_sync`)
`o_soft_clr`	1	Out	mọi module	Xung xóa đồng bộ 1 chu kỳ (RST-03)
`o_soft_busy`	1	Out	RF	Đang trong chuỗi soft reset (RST-07)
Register File → FSM				
`i_start_pulse`	1	In	RF	Một tác vụ vừa được chấp nhận
`i_start_kind`	2	In	RF	Bit 0: START, bit 1: START_CHAIN đã được ghi 1
`i_soft_req`	1	In	RF	Yêu cầu soft reset (xung)
`i_cfg_dim` … `i_cfg_qshift`	8 × 32	In	RF	Bản chụp: `MATRIX_DIM`, `BASE_ADDR_IN/WEIGHT/OUT`, `POST_CTRL`, `BIAS_ADDR`, `QUANT_M0`, `QUANT_SHIFT` (đủ 32 bit)
`i_cfg_desc_ptr`	32	In	RF	Bản chụp `DESC_PTR`
FSM → Register File				
`o_done_pulse`	1	Out	RF	Tác vụ kết thúc thành công
`o_err_pulse`, `o_err_code`	1, 4	Out	RF	Tác vụ kết thúc với lỗi, mã lỗi
`o_perf_load`, `o_perf_comp`, `o_perf_store`	1 × 3	Out	RF	Mức: đang chạy LOAD_*, COMPUTE, STORE_C
`o_err_desc_addr`	32	Out	RF	`ERR_DESC_ADDR`
`o_desc_done_cnt`	32	Out	RF	`DESC_DONE_CNT`
Lệnh kênh đọc DMA				
`o_rd_start`	1	Out	DMA	Xung: bắt đầu lệnh; các port `o_rd_*` khác hợp lệ ở chu kỳ này
`o_rd_dst`	2	Out	DMA	0: A, 1: B, 2: BIAS, 3: descriptor (FETCH)
`o_rd_base`	32	Out	DMA	Địa chỉ byte của hàng đầu
`o_rd_rows`, `o_rd_rwords`	6, 6	Out	DMA	Số hàng 1 … 32; số word mỗi hàng 1 … 32
`o_rd_stride`	16	Out	DMA	Khoảng cách byte giữa hai hàng ($\geq 4 \cdot$ `rwords`)
`o_rd_bank`	1	Out	DMA	Bank đích
`o_rd_abort`	1	Out	DMA	Mức: yêu cầu abort (DMA-10)
`i_rd_busy`, `i_rd_done`, `i_rd_error`	1 × 3	In	DMA	Kênh bận; xung xong; xung lỗi
`i_desc_valid`, `i_desc_idx`, `i_desc_data`	1, 4, 32	In	DMA	Một word descriptor vừa đọc về
Lệnh kênh ghi DMA				
`o_wr_start`	1	Out	DMA	Xung: bắt đầu STORE_C
`o_wr_fmt`	1	Out	DMA	0: INT32, 1: INT8
`o_wr_base`, `o_wr_rows`, `o_wr_rwords`, `o_wr_stride`, `o_wr_bank`	32, 6, 6, 16, 1	Out	DMA	Như kênh đọc
`o_wr_abort`	1	Out	DMA	Mức: yêu cầu abort
`i_wr_busy`, `i_wr_done`, `i_wr_error`	1 × 3	In	DMA	Như kênh đọc
Array, VPU và scratchpad phía tính				
`o_pe_start`	1	Out	Array	Xung bắt đầu COMPUTE
`i_pe_done`, `i_pe_busy`	1, 1	In	Array	Xung xong (đã qua VPU vào buffer C); mức bận
`o_comp_dim`	32	Out	Array	{8'b0, K, N, M} của job đang tính (bố cục `MATRIX_DIM`)

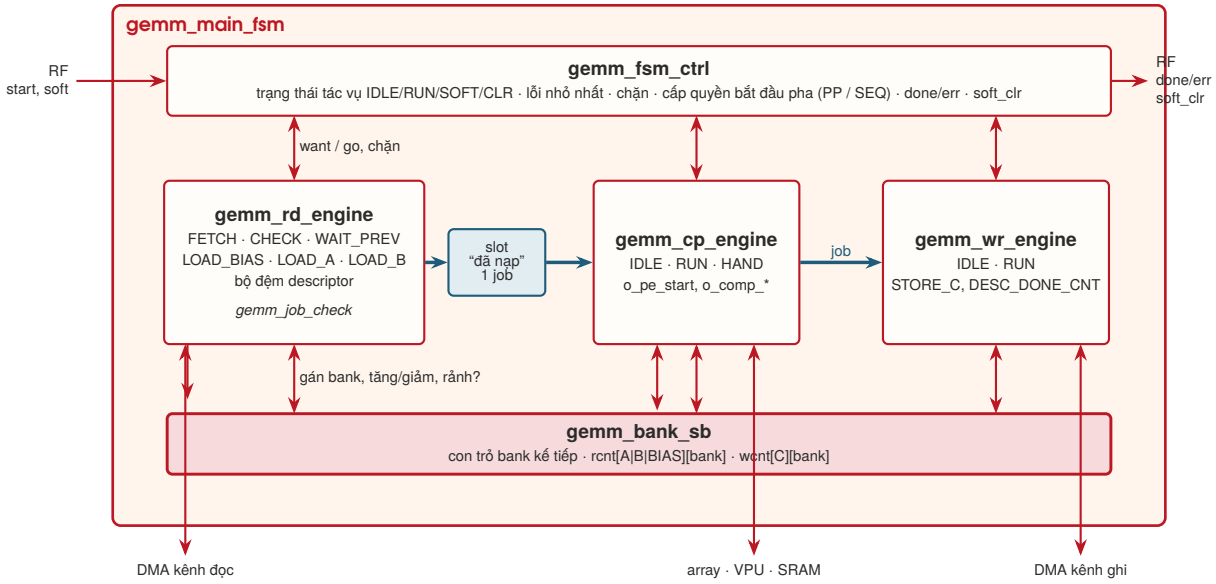
Bảng 3.1: Port của `gemm_main_fsm` (tiếp theo)

Port	Rộng	Chiều	Nối với	Mô tả
`o_comp_post_en`, `o_comp_relu_en`	1, 1	Out	VPU	
`o_comp_qm0`, `o_comp_qshift`	32, 5	Out	VPU	
`o_comp_bank_a`, `_b`, `_bias`, `_c`	1 × 4	Out	SRAM	Bank mà array/VPU đang dùng
`o_comp_active`	1	Out	SRAM	Mức: COMPUTE đang chạy (cho assertion)

Bảng 3.2: Giao tiếp

ID	Yêu cầu
FSM-I01	Mọi port `o_*` lấy từ thanh ghi, trừ `o_rd_abort` và `o_wr_abort` (mức, cũng từ thanh ghi trạng thái)
FSM-I02	Lệnh DMA: FSM chỉ bật `o_rd_start` khi kênh đọc rảnh (`i_rd_busy` = 0 và không có lệnh nào đang chờ `i_rd_done`/`i_rd_error`); tương tự kênh ghi. Các port tham số chỉ cần hợp lệ ở chu kỳ `o_*_start` = 1; DMA tự chốt
FSM-I03	`o_comp_dim`, `o_comp_post_en`, `o_comp_relu_en`, `o_comp_qm0`, `o_comp_qshift`, `o_comp_bank_*` hợp lệ từ chu kỳ `o_pe_start` = 1 tới hết chu kỳ `i_pe_done` = 1 và không đổi trong khoảng đó (SA-06, VPU-I05)
FSM-I04	`o_pe_start` chỉ khi `i_pe_busy` = 0. Mỗi `o_pe_start` có đúng một `i_pe_done` (SA-07)
FSM-I05	`o_done_pulse` và `o_err_pulse` không bao giờ cùng chu kỳ; mỗi `i_start_pulse` có tối đa một trong hai; không có cái nào cho tác vụ bị soft reset hủy (FSM-04, RST-08)

4 Vi kiến trúc



Hình 4.1: Cấu trúc bên trong `gemm_main_fsm`

Bảng 4.1: Module con

Module	Instance	Vai trò
`gemm_fsm_ctrl`	`u_ctrl`	Trạng thái tác vụ; xử lý `i_start_kind`; bản ghi lỗi; tín hiệu chặn; cấp quyền bắt đầu pha; done/error; soft reset; `ERR_DESC_ADDR`
`gemm_job_check`	`u_chk` (trong engine Đọc)	Tổ hợp: dựng job từ bản chụp hoặc descriptor; kiểm REG-12 hoặc DSC-03; tính stride hiệu dụng
`gemm_rd_engine`	`u_rd`	Engine Đọc: FETCH, CHECK, chờ WAIT_PREV, LOAD_*, đẩy job vào slot
`gemm_cp_engine`	`u_cp`	Engine Tính: lấy job từ slot, chạy COMPUTE, giữ job tới khi engine Ghi nhận
`gemm_wr_engine`	`u_wr`	Engine Ghi: STORE_C; `DESC_DONE_CNT`; báo job cuối
`gemm_bank_sb`	`u_sb`	Chọn bank theo bảng 7-4; bộ đếm chiếm dụng bank

4.1 Job

Một job là struct `gemm_pkg::job_t`. Engine Đọc giữ một job, slot giữ một job, engine Tính và engine Ghi mỗi bên giữ một job; tối đa bốn job cùng tồn tại.

Bảng 4.2: Trường của `job_t`

Trường	Rộng	Nội dung
`idx`	32	Số thứ tự trong chuỗi, 0, 1, 2, …; luôn 0 ở chế độ đơn
`daddr`	32	Địa chỉ descriptor (cho `ERR_DESC_ADDR`); 0 ở chế độ đơn
`m, n, k`	6×3	M, N, K (1 … 32)
`a, b, c, bias`	32×4	Địa chỉ byte của A, B, C/Y, bias
`lda, ldb, ldc`	16×3	Stride hiệu dụng (byte), đã thay 0 bằng độ dài hàng
`qm0, qsh`	32, 5	M_0, sh
`post, relu, reuse, waitp, last`	1×5	Cờ
`next`	32	`NEXT_PTR` (chỉ engine Đọc dùng)
`ba, bb, bbias, bc`	1×4	Bank của A, B, BIAS, C (gán ở CHECK)

5 Dựng và kiểm tra job

5.1 Chế độ đơn

Bảng 5.1: Job từ bản chụp (`gemm_job_check`, mode = đơn)

Trường	Giá trị
`m, n, k`	`i_cfg_dim[7:0]`, `[15:8]`, `[23:16]` (6 bit thấp)
`a, b, c, bias`	`i_cfg_a_addr`, `i_cfg_b_addr`, `i_cfg_c_addr`, `i_cfg_bias_addr`
`lda, ldb, ldc`	K; N; N nếu `post`, ngược lại 4N
`post, relu`	`i_cfg_post_ctrl[0]`, `[1]`
`qm0, qsh`	`i_cfg_qm0`, `i_cfg_qshift[4:0]`
`idx, daddr, reuse, waitp, last`	0, 0, 0, 0, 1

Job hợp lệ khi và chỉ khi thỏa REG-12 của spec hệ thống (kiểm trên đủ 32 bit của bản chụp).

5.2 Chế độ chuỗi

Engine Đọc nhận 16 word descriptor qua `i_desc_*` và lưu word 0 … 10 vào bộ đệm. Word 11 (`TAG`) được bỏ qua; word 12 … 15 chỉ cần OR dồn để kiểm bằng 0.

Bảng 5.2: Job từ descriptor

Trường	Giá trị (wn = word thứ n)
`m, n, k`	w0[7:0], w0[15:8], w0[23:16]
`a, b, c`	w1, w2, w3
`lda, ldb`	w4[15:0], w4[31:16]; 0 → K, N
`ldc`	w5[15:0]; 0 → N nếu `post`, ngược lại 4N
`last, waitp, reuse, post, relu`	w6[0], w6[1], w6[2], w6[3], w6[4]
`next, bias, qm0, qsh`	w7, w8, w9, w10[4:0]
`idx, daddr`	Bộ đếm descriptor của engine Đọc; con trỏ dùng để FETCH

Bảng 5.3: Kiểm tra descriptor (`gemm_job_check`, mode = chuỗi)

ID	Yêu cầu
CHK-01	Hợp lệ khi và chỉ khi mọi điều sau đúng (DSC-03): $1 \le M \le 32$; N, K $\in$ {4, 8, …, 32}; w0[31:24] = 0; w5[31:16] = 0; w6[31:5] = 0; w10[31:5] = 0; w12 … w15 = 0
CHK-02	`a`[1:0] = 0 và `c`[1:0] = 0; `b`[1:0] = 0 nếu `reuse` = 0; `bias`[1:0] = 0 nếu `post` = 1
CHK-03	Mỗi stride thô (w4[15:0], w4[31:16], w5[15:0]) bằng 0, hoặc là bội của 4 và không nhỏ hơn độ dài hàng: K cho A; N cho B; 4N cho C khi `post` = 0, N khi `post` = 1
CHK-04	`relu` = 1 kéo theo `post` = 1
CHK-05	`reuse` = 1 chỉ khi `idx` > 0 và (K, N) bằng (K, N) của job trước trong chuỗi (thanh ghi `prev_kn` của engine Đọc)
CHK-06	`last` = 0 kéo theo `next`[5:0] = 0 (DSC-01). Vi phạm được tính là lỗi của chính descriptor này
CHK-07	`DESC_PTR`[5:0] $\neq$ 0 được kiểm ở `gemm_fsm_ctrl` khi bắt đầu chuỗi: lỗi mã 1 với f = 0, `ERR_DESC_ADDR` = `DESC_PTR`, không truy cập bus

6 Bank và cấp quyền bắt đầu pha

6.1 Chọn bank

Bảng 6.1: `gemm_bank_sb`: chọn bank (bảng 7-4 spec hệ thống)

ID	Yêu cầu
BNK-01	Thanh ghi con trỏ `nxt_a`, `nxt_c`, `nxt_bias`, `nxt_b`, `prev_b`: về 0 khi chấp nhận tác vụ, khi `rst_n` và khi `soft_clr`
BNK-02	Ở chu kỳ CHECK của job hợp lệ: `ba=nxt_a`, `bc=nxt_c`; `bbias=nxt_bias` nếu `post` (ngược lại 0); `bb=prev_b` nếu `reuse`, ngược lại `bb=nxt_b` và `prev_b<=nxt_b`
BNK-03	Với NB = 2, cùng cạnh clock đó đảo `nxt_a`, `nxt_c`; đảo `nxt_bias` nếu `post`; đảo `nxt_b` nếu `reuse` = 0. Với NB = 1 mọi bank là 0

6.2 Bộ đếm chiếm dụng

Bảng 6.2: Bộ đếm của `gemm_bank_sb`

Bộ đếm	Rộng	Tăng khi	Giảm khi
`rcnt[A][b]`	2	Engine Đọc bắt đầu LOAD_A vào bank b	COMPUTE của job có `ba` = b xong
`rcnt[B][b]`	2	LOAD_B vào bank b, hoặc job `reuse` = 1 có `bb` = b đi qua bước B	COMPUTE của job có `bb` = b xong
`rcnt[BIAS][b]`	2	LOAD_BIAS vào bank b	COMPUTE của job `post` = 1 có `bbias` = b xong
`wcnt[C][b]`	1	COMPUTE của job có `bc` = b bắt đầu	STORE_C của job đó xong OKAY

Bảng 6.3: Điều kiện bank (PP-03, PP-04)

ID	Yêu cầu
SB-01	LOAD_X vào bank b chỉ được bắt đầu khi `rcnt[X][b]` = 0: mọi job đọc bank đó đã tính xong. Job `reuse` = 1 không chờ (dữ liệu B của bank đó vẫn còn, vì chưa có LOAD_B nào vào bank ấy kể từ job trước)
SB-02	COMPUTE vào bank C b chỉ được bắt đầu khi `wcnt[C][b]` = 0: STORE_C cuối cùng đọc bank đó đã xong
SB-03	Bộ đếm không bao giờ tràn hay âm (assertion). `rcnt` tối đa 3 vì tối đa ba job đã gán bank mà chưa tính xong (engine Đọc, slot, engine Tính)
SB-04	Khi tác vụ kết thúc do lỗi, bộ đếm có thể khác 0; chúng được xóa khi chấp nhận tác vụ kế tiếp hoặc khi `soft_clr`

6.3 Cấp quyền bắt đầu pha

Mỗi engine tính tín hiệu `want` từ trạng thái đầu chu kỳ: `want_s` (Ghi), `want_c` (Tính), `want_r` (Đọc: FETCH hoặc một LOAD). `gemm_fsm_ctrl` trả `go`.

$$\begin{aligned}
&\text{PP } (NB=2): && go_x = want_x \wedge \neg inh \\
&\text{SEQ } (NB=1): && go_s = want_s \wedge \neg act \wedge \neg inh, \quad go_c = want_c \wedge \neg act \wedge \neg want_s \wedge \neg inh, \\
& && go_r = want_r \wedge \neg act \wedge \neg want_s \wedge \neg want_c \wedge \neg inh
\end{aligned}$$

Trong đó `act` = (engine Đọc đang chờ một lệnh DMA) $\vee$ (engine Tính ở RUN) $\vee$ (engine Ghi ở RUN); `inh` = tác vụ không ở trạng thái RUN (gồm SOFT, CLR). Ưu tiên Ghi > Tính > Đọc làm job cũ ra trước và cho đúng lịch tuần tự của hình 7-2a.

Bảng 6.4: Chặn sau lỗi (DSC-08)

ID	Yêu cầu
BLK-01	`gemm_fsm_ctrl` cấp `blk_v`, `blk_idx` tổ hợp: bản ghi lỗi hiện có, gộp với lỗi đến trong chính chu kỳ này (`i_rd_error`, `i_wr_error`, kết quả CHECK sai), lấy chỉ số nhỏ nhất
BLK-02	Mọi `want` của job có `idx` $\geq$ `blk_idx` bằng 0 khi `blk_v` = 1. Nhờ gộp lỗi cùng chu kỳ, không pha nào của descriptor $\geq$ f bắt đầu, kể cả ở đúng chu kỳ lỗi được báo
BLK-03	Pha đã bắt đầu thì chạy tới hết (không abort khi lỗi). Kết quả của chúng không được ghi ra DRAM vì STORE_C của job $\geq$ f không bao giờ bắt đầu

7 Engine

7.1 Lệnh DMA

Bảng 7.1: Tham số lệnh DMA của mỗi pha

Pha	dst / fmt	base	rows	rwords	stride	bank
FETCH	3 (desc)	con trỏ	1	16	64	0
LOAD_BIAS	2 (BIAS)	`bias`	1	N	4N	`bbias`
LOAD_A	0 (A)	`a`	M	K/4	`lda`	`ba`
LOAD_B	1 (B)	`b`	K	N/4	`ldb`	`bb`
STORE_C	`post`	`c`	M	`post` ? N/4 : N	`ldc`	`bc`

7.2 Engine Đọc

Bảng 7.2: Trạng thái của `gemm_rd_engine`

Trạng thái	Làm gì	Sang
R_IDLE	Chờ	R_START khi ctrl khởi động tác vụ
R_START	Chế độ đơn: nạp job từ bản chụp. Chuỗi: con trỏ ← `DESC_PTR`, idx ← 0	R_CHECK (đơn) / R_FETCH (chuỗi)
R_FETCH	`want_r` = 1. Khi `go_r`: phát FETCH	R_FWAIT
R_FWAIT	Nhận word descriptor; chờ `i_rd_done` / `i_rd_error`	R_CHECK / R_STOP (lỗi mã 4)
R_CHECK	1 chu kỳ: `gemm_job_check`; nếu hợp lệ, gán bank (BNK-02)	R_WAITP / R_STOP (lỗi mã 1)
R_WAITP	Nếu `waitp` = 1: chờ `desc_done_cnt==idx` (mọi STORE_C trước đã xong, DSC-05)	R_LBIAS nếu `post`, ngược lại R_LA
R_LBIAS, R_LA, R_LB	`want_r` = (`rcnt[X][bank]` = 0). Khi `go_r`: `rcnt[X][bank]++`, phát lệnh theo bảng 7.1	R_LWAIT
R_LWAIT	Chờ `i_rd_done` / `i_rd_error`	tiếp theo thứ tự BIAS → A → (B hoặc RB) → PUSH / R_STOP (lỗi mã 2)
R_RB	Job `reuse`: `rcnt[B][bb]++`, không phát lệnh	R_PUSH
R_PUSH	Khi slot trống: slot ← job. Chuỗi và `last` = 0: `prev_kn` ← (K, N), con trỏ ← `next`, idx++	R_FETCH; hoặc R_STOP nếu `last` = 1 hoặc chế độ đơn
R_STOP	Không làm gì tới hết tác vụ	—

Bảng 7.3: Engine Đọc

ID	Yêu cầu
RDE-01	Ở mọi trạng thái chờ (R_FETCH, R_WAITP, R_LBIAS, R_LA, R_LB, R_PUSH): nếu job bị chặn (BLK-02) thì sang R_STOP, không đẩy job vào slot
RDE-02	Engine Đọc làm xong mọi LOAD của job d rồi mới FETCH job d+1 (PP-02). `WAIT_PREV` không chặn FETCH và CHECK (PP-06)
RDE-03	Bộ đệm descriptor ghi word `i_desc_idx` khi `i_desc_valid`. Dữ liệu của FETCH bị lỗi không được dùng
RDE-04	`o_perf_load` = 1 khi ở R_LWAIT (không tính FETCH)

7.3 Engine Tính và engine Ghi

Bảng 7.4: Trạng thái của `gemm_cp_engine` và `gemm_wr_engine`

Trạng thái	Làm gì	Sang
C_IDLE	`want_c` = slot có job ∧ `wcnt[C][slot.bc]` = 0. Khi `go_c`: job ← slot, slot trống, `wcnt++`, `o_pe_start` = 1 ở chu kỳ kế tiếp	C_RUN
C_RUN	`o_comp_active` = `o_perf_comp` = 1. Khi `i_pe_done`: giảm `rcnt[A][ba]`, `rcnt[B][bb]`, và `rcnt[BIAS][bbias]` nếu `post`	C_HAND
C_HAND	Giữ job (và bank C) cho engine Ghi	C_IDLE khi engine Ghi nhận job
S_IDLE	`want_s` = engine Tính ở C_HAND. Khi `go_s`: job ← job của engine Tính, phát STORE_C ở chu kỳ kế tiếp	S_RUN
S_RUN	`o_perf_store` = 1. `i_wr_done`: `wcnt[C][bc]--`; ở chế độ chuỗi `desc_done_cnt++`; nếu `last` = 1, báo job cuối xong. `i_wr_error`: lỗi mã 3	S_IDLE

VÌ SAO ENGINE TÍNH GIỮ JOB Ở C_HAND KHÔNG LÀM CHẬM
COMPUTE(d+1) dùng bank C của job d−1, nên theo PP-04 chỉ bắt đầu được khi STORE_C(d−1) xong. Đúng lúc đó engine Ghi rảnh và nhận job d, nên engine Tính về C_IDLE và có thể bắt đầu d+1 ngay chu kỳ sau. Với NB = 1, lịch vốn đã tuần tự.

8 Điều khiển tác vụ

Bảng 8.1: Trạng thái của `gemm_fsm_ctrl`

Trạng thái	Làm gì	Sang
T_IDLE	Chờ. `i_start_pulse`: xóa bản ghi lỗi, `desc_done_cnt`, `err_desc_addr`, con trỏ bank, bộ đếm; xét `i_start_kind` (bảng 8.2)	T_RUN. `i_soft_req`: T_CLR
T_RUN	Engine chạy. Kiểm điều kiện kết thúc (TSK-01 … TSK-03 dưới)	T_IDLE khi done/err; T_SOFT khi `i_soft_req`
T_SOFT	`inh` = 1; bật `o_rd_abort` khi kênh đọc bận, `o_wr_abort` khi kênh ghi bận; không báo done/err	T_CLR khi không engine nào chờ lệnh DMA
T_CLR	`o_soft_clr` = 1 trong 1 chu kỳ; mọi thanh ghi của FSM về giá trị reset	T_IDLE

Bảng 8.2: Xử lý `i_start_kind`

Giá trị	Hành vi
2'b01 (START)	Chế độ đơn: engine Đọc vào R_START
2'b10 (START_CHAIN)	`HAS_CHAIN` = 0: lỗi mã 5. `DESC_PTR`[5:0] ≠ 0: lỗi mã 1 (CHK-07). Ngược lại: chế độ chuỗi
2'b11	Lỗi mã 1 (REG-05). Không engine nào chạy, không truy cập bus

Bảng 8.3: Kết thúc tác vụ

ID	Yêu cầu
TSK-F01	Bản ghi lỗi (`err_v`, `err_idx`, `err_code`, `err_addr`) chỉ được thay khi lỗi mới có idx nhỏ hơn (DSC-08). Mã: FETCH → 4, CHECK → 1, LOAD_* → 2, STORE_C → 3. `err_addr` = `daddr` của job lỗi; = con trỏ đang FETCH với lỗi mã 4; = `DESC_PTR` với CHK-07
TSK-F02	DONE: ở T_RUN, engine Ghi báo job có `last` = 1 xong OKAY và `err_v` = 0 → `o_done_pulse` ở chu kỳ kế tiếp

Bảng 8.3: Kết thúc tác vụ (tiếp theo)

ID	Yêu cầu
TSK-F03	ERROR: ở T_RUN, `err_v` = 1 và hệ yên: engine Đọc ở R_STOP và không chờ DMA; engine Tính không ở C_RUN; slot trống hoặc có idx $\geq$ f; engine Tính không giữ ở C_HAND một job idx $<$ f; engine Ghi ở S_IDLE $\rightarrow$ `o_err_pulse` với `o_err_code` = `err_code` ở chu kỳ kế tiếp; `o_err_desc_addr` $\leftarrow$ `err_addr` (chỉ ở chế độ chuỗi)
TSK-F04	Khi đó `o_desc_done_cnt` = f tự nhiên, vì mọi job $<$ f đã ghi xong và không job $\geq$ f nào được ghi
TSK-F05	`o_err_desc_addr` và `o_desc_done_cnt` bằng 0 ở chế độ đơn và khi `HAS_CHAIN` = 0

Bảng 8.4: Soft reset (RST-05 … RST-09, PP-09)

ID	Yêu cầu
SFT-01	`o_soft_busy` = 1 ở T_SOFT và T_CLR
SFT-02	Ở T_SOFT: không pha mới nào bắt đầu; lệnh DMA đang chạy kết thúc bằng done hoặc error như thường (DMA-10); engine vẫn nhận các xung đó để rời trạng thái chờ, nhưng không cập nhật bản ghi lỗi hay báo done
SFT-03	COMPUTE đang chạy không được chờ: `o_soft_clr` hủy array và VPU (RST-06)
SFT-04	`i_soft_req` ở T_IDLE: T_CLR ở chu kỳ kế tiếp. `i_soft_req` ở T_SOFT hoặc T_CLR: bỏ qua (RST-07)
SFT-05	Nếu `i_soft_req` đến cùng chu kỳ với `i_rd_done` hoặc `i_wr_done`, kênh đó coi như đã rảnh (RST-09)

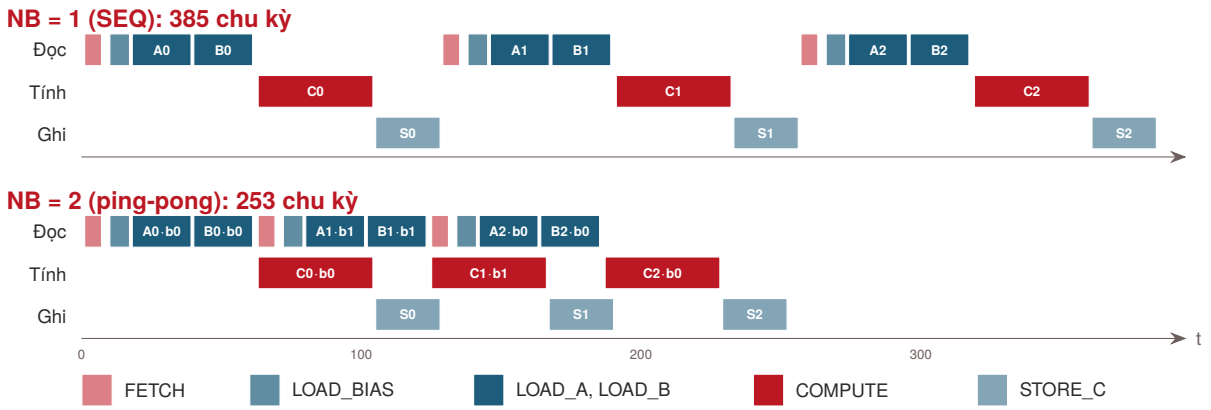
9 Timing

Bảng 9.1: Độ trễ cố định của FSM

Sự kiện	Chu kỳ
`i_start_pulse` $\rightarrow$ engine Đọc ở R_START	+1
R_CHECK	1 chu kỳ
`go_*` = 1 $\rightarrow$ `o_rd_start` / `o_wr_start` / `o_pe_start` = 1	+1
`i_*_done` $\rightarrow$ trạng thái kế tiếp của engine	+1
Điều kiện DONE / ERROR $\rightarrow$ `o_done_pulse` / `o_err_pulse`	+1
`i_soft_req` ở T_IDLE $\rightarrow$ `o_soft_clr`	+1

9.1 Ví dụ lịch chạy

Chuỗi ba descriptor giống nhau (M = N = K = 16, `POST_EN` = 1), thời lượng giả định của mỗi lệnh: FETCH 5, LOAD_BIAS 6, LOAD_A 20, LOAD_B 20, COMPUTE 40, STORE_C 22 chu kỳ. Hình lấy đúng từ `gemm_sched_model.py`.



Hình 9.1: Lịch của cùng một chuỗi khi NB = 1 và NB = 2. Với NB = 2, FETCH và LOAD của descriptor d+1 chạy trong lúc COMPUTE(d) và STORE_C(d−1); tổng thời gian giảm từ 385 xuống 253 chu kỳ

10 Reset, hiện thực, kiểm chứng

Bảng 10.1: Reset

ID	Yêu cầu
RST-F01	`rst_n` = 0 hoặc T_CLR: mọi engine về trạng thái IDLE, slot trống, bản ghi lỗi, con trỏ bank, bộ đếm, `desc_done_cnt`, `err_desc_addr`, mọi xung đầu ra về 0. Trường dữ liệu của job không bắt buộc reset

Bảng 10.2: Hiện thực

ID	Yêu cầu
IMP-01	SystemVerilog tổng hợp được; FSM viết theo kiểu hai khối (`always_ff` trạng thái, `always_comb` trạng thái kế); không có latch
IMP-02	`job_t` khai báo trong `gemm_pkg` là `structpacked`; công cụ tổng hợp tự bỏ trường không dùng ở mỗi engine
IMP-03	Với `HAS_CHAIN` = 0, logic FETCH, bộ đệm descriptor, CHK-01 … CHK-07 và `desc_done_cnt` được loại bằng `generate`. Với NB = 1, bộ đếm và con trỏ bank được loại, các điều kiện bank luôn đúng trừ quy tắc SEQ
IMP-04	Mọi so sánh idx dùng 32 bit không dấu

Bảng 10.3: Assertion bắt buộc (SVA)

ID	Yêu cầu
AS-01	`o_rd_start` → kênh đọc rảnh; `o_wr_start` → kênh ghi rảnh; `o_pe_start` → `!i_pe_busy`
AS-02	NB = 1: tại mọi chu kỳ có tối đa một trong {engine Đọc chờ DMA, C_RUN, S_RUN}
AS-03	Không bộ đếm nào tràn hay âm (SB-03)
AS-04	LOAD_X bắt đầu vào bank b → không job nào ở slot, C_RUN hay C_HAND chưa tính xong có bank X = b (trừ job reuse)
AS-05	STORE_C bắt đầu theo thứ tự idx tăng dần 1
AS-06	`o_done_pulse` ∧ `o_err_pulse` không bao giờ cùng lúc; mỗi `i_start_pulse` có tối đa một trong hai
AS-07	`o_comp_*` ổn định trong khi C_RUN

KIỂM CHỨNG BẰNG MÔ HÌNH

`gemm_sched_model.py` chạy 1 200 chuỗi ngẫu nhiên 1 – 7 descriptor (stride, `REUSE_B`, `WAIT_PREV` với descriptor đọc kết quả của descriptor trước, descriptor sai, lỗi AXI tiêm ngẫu nhiên ở FETCH/LOAD/STORE) với cả NB = 1 và NB = 2, và 300 tác vụ chế độ đơn. Mọi bên đọc lấy dữ liệu ở đầu và cuối pha và phải thấy giống nhau, nên ghi đè sai bank bị phát hiện. Kết quả: DRAM khớp thực thi tuần tự ở mọi chuỗi DONE; với lỗi, `DESC_DONE_CNT` = f, `ERR_DESC_ADDR` đúng, không pha nào của job $\geq$ f bắt đầu sau lỗi; NB = 1 không có hai pha chồng nhau. Mô hình đã bắt hai lỗi quy tắc trong quá trình thiết kế (đường `REUSE_B` và lỗi cùng chu kỳ), dẫn tới bước R_RB và BLK-01.

A Nối dây nội bộ trong `gemm_accel_top`

Bảng A.1: Kết nối giữa các module (bản full option)

Từ	Tới	Tín hiệu
RF `o_start_pulse`, `o_start_kind`, `o_soft_req`, `o_cfg_*`	FSM `i_*` cùng tên	Mục 3
FSM `o_done_pulse`, `o_err_*`, `o_perf_*`, `o_soft_busy`, `o_err_desc_addr`, `o_desc_done_cnt`	RF `i_*` cùng tên	
FSM `o_soft_clr`	`i_soft_clr` của RF, DMA, SRAM, Array, VPU	
FSM `o_rd_*`, `o_wr_*`	DMA `i_rd_*`, `i_wr_*`	Lệnh hai kênh
DMA `o_rd_busy/done/error`, `o_desc_*`, `o_wr_busy/done/error`	FSM `i_*` cùng tên	
DMA `o_spm_wr_*`, `o_spm_rd_*`	SRAM `i_dw_*`, `i_dr_*`	Spec DMA, spec SRAM
SRAM `o_dw_ready`, `o_dr_ready`, `o_dr_rvalid`, `o_dr_rdata`	DMA `i_spm_*`	
FSM `o_pe_start`; Array `o_pe_done`, `o_busy`	Array `i_pe_start`; FSM `i_pe_done`, `i_pe_busy`	
FSM `o_comp_dim`	Array `i_matrix_dim`	
FSM `o_comp_post_en/relu_en/qm0/qshift`	VPU `i_post_en/relu_en/quant_m0/quant_shift`	
FSM `o_comp_bank_*`, `o_comp_active`	SRAM `i_comp_bank_*`, `i_comp_active`	
Array `o_a_rd_*`, `o_b_rd_*` ↔ SRAM	SRAM `i_a_rd_*`, `i_b_rd_*`, `o_a_rd_data`, `o_b_rd_data`	Spec array mục 3
Array `o_row_*`	VPU `i_row_*`	Spec VPU mục 3
VPU `o_bias_rd_*`, `o_c_wr_*` ↔ SRAM	SRAM `i_bias_rd_*`, `o_bias_rd_data`, `i_c_wr_*`	
top `s_axi_*`	RF	17 tín hiệu
DMA `m_axi_*`	top	25 tín hiệu